

Divide and Conquer

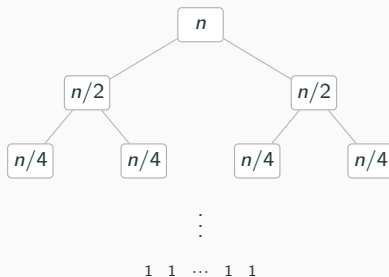
Tutorial

Algorithm Analysis and Design

Plan

- | | |
|---------------------------------------|----------|
| 1. Solving recurrences | 2.5 |
| 2. Fast Fourier transform | 2.7, 2.8 |
| 3. Matrix squaring vs. multiplication | 2.27 |
| 4. Closest pair of points | 2.32 |
| 5. Polynomial interpolation | 2.10 |

Solving recurrences - the recursion tree



$$1 \times n = n$$

$$2 \times \frac{n}{2} = n$$

$$4 \times \frac{n}{4} = n$$

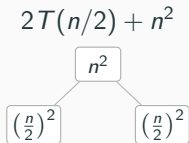
$\vdots$

$$n \times 1 = n$$

$T(n) = 2T(n/2) + n$: label each node with its **own** work, then sum level by level.

$$\underbrace{\log_2 n}_{\text{levels}} \times \underbrace{n}_{\text{per level}} = \Theta(n \log n)$$

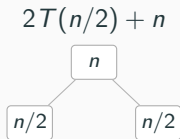
Solving recurrences - three shapes



$n^2, \frac{n^2}{2}, \frac{n^2}{4}, \dots$

shrinks $\rightarrow$ root wins

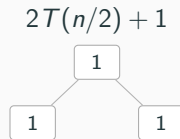
$$\Theta(n^2)$$



$n, n, n, \dots$

equal $\rightarrow$ nobody wins

$$\Theta(n \log n)$$



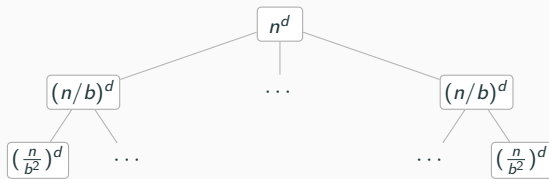
$1, 2, 4, \dots$

grows $\rightarrow$ leaves win

$$\Theta(n)$$

Same branching, different local work. These three pictures are the master theorem.

Solving recurrences - setting up the sum



level 0: 1 node

level 1: a nodes

level 2: a^2 nodes

$\vdots$

$$\text{level } j = a^j \left(\frac{n}{b^j} \right)^d = n^d r^j, \quad r = \frac{a}{b^d} \quad \Rightarrow \quad T(n) = n^d \sum_{j=0}^{\log_b n} r^j$$

geometric series with ratio r ?

Solving recurrences - which shape are you in?

	$r < 1$	$r = 1$	$r > 1$
levels	shrinking	equal	growing
dominated by	the root	nobody	the leaves
condition	$a < b^d$	$a = b^d$	$a > b^d$
equivalently	$d > \log_b a$	$d = \log_b a$	$d < \log_b a$
$T(n)$	$\Theta(n^d)$	$\Theta(n^d \log n)$	$\Theta(n^{\log_b a})$

Geometric series: $\Theta(\text{first term}) / \Theta(\text{number of terms}) / \Theta(\text{last term})$.

Solving recurrences - master theorem

For $T(n) = a T(n/b) + \Theta(n^d)$, compare d with $\log_b a$:

$$T(n) = \begin{cases} \Theta(n^d) & d > \log_b a \\ \Theta(n^d \log n) & d = \log_b a \\ \Theta(n^{\log_b a}) & d < \log_b a \end{cases}$$

$n^{\log_b a}$ is the number of leaves. Root work vs. leaf work.

Solving recurrences - two families

Divide $T(n/b)$ (a)–(f)

Master theorem applies.

Subtract $T(n-1)$, $T(\sqrt{n})$ (g)–(k)

Unroll and sum.

(a) $\Theta(n^{\log_3 2})$	(d) $\Theta(n^2 \log n)$	(g) $\Theta(n)$	(j) $\Theta(2^n)$
(b) $\Theta(n^{\log_4 5})$	(e) $\Theta(n^3 \log n)$	(h) $\Theta(n^{c+1})$	(k) $\Theta(\log \log n)$
(c) $\Theta(n \log n)$	(f) $\Theta(n^{3/2} \log n)$	(i) $\Theta(c^n)$	

Solving recurrences - two edge cases

(f) Driving term is not $\Theta(n^d)$ — sum the tree.

$$T(n) = 49 T(n/25) + n^{3/2} \log n \quad \text{level } j \leq \left(\frac{49}{125}\right)^j n^{3/2} \log n$$

Geometric, ratio $< 1 \Rightarrow$ root dominates: $\Theta(n^{3/2} \log n)$.

(k) Size shrinks by a root — change variable.

$$T(n) = T(\sqrt{n}) + 1 \quad \xrightarrow{n=2^m} \quad S(m) = S(m/2) + 1 \quad \Rightarrow \quad \Theta(\log \log n)$$

Fast Fourier transform - motivation

Goal: multiply two polynomials.

$$C(x) = A(x) \cdot B(x), \quad \deg A, \deg B < n$$

Naive way: convolve the coefficients,

$$c_k = \sum_{i+j=k} a_i b_j \quad \implies \quad \Theta(n^2) \text{ work.}$$

Idea: change representation. Instead of coefficients, describe a polynomial by its **values** at n points $x_0, \dots, x_{n-1}$:

$$A \longleftrightarrow (A(x_0), \dots, A(x_{n-1}))$$

Then multiplication is just **pointwise**:

$$C(x_i) = A(x_i) \cdot B(x_i) \quad \implies \quad \Theta(n) \text{ work, once we have the values.}$$

So **which points x_i should we evaluate at?**

Fast Fourier transform - roots of unity 2.7

$$\omega = e^{2\pi i/n}, \quad \text{roots} = 1, \omega, \omega^2, \dots, \omega^{n-1}$$

Sum (geometric series, $n \geq 2$):

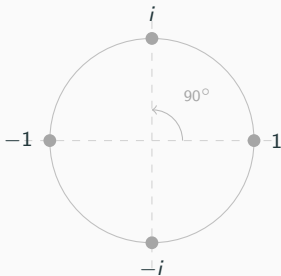
$$\sum_{k=0}^{n-1} \omega^k = \frac{\omega^n - 1}{\omega - 1} = 0$$

Product:

$$\prod_{k=0}^{n-1} \omega^k = \omega^{n(n-1)/2} = (-1)^{n-1} = \begin{cases} +1 & n \text{ odd} \\ -1 & n \text{ even} \end{cases}$$

Sum = 0 is what makes the inverse transform work.

Fast Fourier transform - the roots, on the circle



For $n = 4$: $\omega = i$, and the roots

$$1, \omega, \omega^2, \omega^3 = 1, i, -1, -i$$

sit at 90° apart on the unit circle.

In general, the n -th roots are spaced $\frac{360^\circ}{n}$ apart — **equally spaced**, which is exactly the symmetry behind

$$1 + \omega + \cdots + \omega^{n-1} = 0.$$

Fast Fourier transform - why it's fast: split by parity

Split the coefficients into even- and odd-indexed halves:

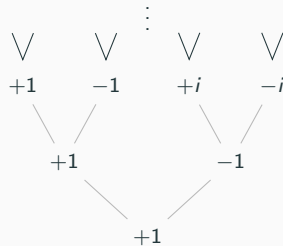
$$A(x) = A_e(x^2) + x A_o(x^2)$$

A_e, A_o have degree $< n/2$. We need A at the n points $\omega^0, \dots, \omega^{n-1}$, so A_e, A_o are needed at the **squared** points $(\omega^k)^2$.

Key fact: ω^2 is a primitive $(n/2)$ -th root of unity, and $(\omega^k)^2 = (\omega^{k+n/2})^2$.

So the n squared points collapse onto only $n/2$ distinct values.

$$T(n) = 2 T(n/2) + O(n) = O(n \log n)$$



Squaring walks roots down to ± 1 — going back up, each level is a choice of square root.

Fast Fourier transform - the transform 2.8

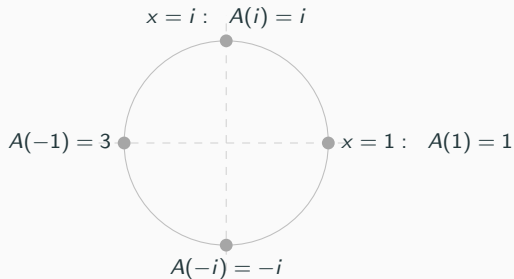
$$\text{FFT}(a_0, \dots, a_{n-1}) = (A(\omega^0), \dots, A(\omega^{n-1})), \quad A(x) = \sum_j a_j x^j$$

For $n = 4$: $\omega = i$, powers $(1, i, -1, -i)$. Inverse: use ω^{-1} , divide by n .

a	$\text{FFT}(a)$	a is the FFT of
$(1, 0, 0, 0)$	$(1, 1, 1, 1)$	$\frac{1}{4}(1, 1, 1, 1)$
$(1, 0, 1, -1)$	$(1, i, 3, -i)$	$\frac{1}{4}(1, -i, 3, i)$

Impulse $\leftrightarrow$ flat. Coefficients $\leftrightarrow$ values, in $O(n \log n)$.

Fast Fourier transform - evaluating directly on the circle



Take $A(x) = 1 + x^2 - x^3$ (coefficients $(1, 0, 1, -1)$).

No recursion needed here — just **plug in each root directly**:

$$A(1) = 1 + 1 - 1 = 1$$

$$A(i) = 1 + i^2 - i^3 = 1 - 1 + i = i$$

$$A(-1) = 1 + 1 + 1 = 3, \quad A(-i) = 1 - 1 - i = -i$$

So $\text{FFT}(1, 0, 1, -1) = (1, i, 3, -i)$ — the value representation.

Matrix multiplication - naive vs. the obvious block recursion

$$C = AB, \quad c_{ij} = \sum_k a_{ik} b_{kj} \quad \implies \quad n^3 \text{ scalar mults, } O(n^3)$$

Block recursion (the obvious way):

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Each $C_{ij} = A_{i1}B_{1j} + A_{i2}B_{2j}$ needs **2 block products** $\Rightarrow$ **8** products of size $n/2$, plus $O(n^2)$ adds.

$$T(n) = 8T(n/2) + O(n^2) = O(n^3) \quad \text{— no gain over naive.}$$

Matrix multiplication - Strassen's seven products

7 products (only +, -, no extra mults):

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22}) B_{11}$$

$$M_3 = A_{11} (B_{12} - B_{22})$$

$$M_4 = A_{22} (B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12}) B_{22}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

Recombine into C:

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

7 multiplications, 18 additions —

trades 1 block mult for extra adds.

$$T(n) = 7T(n/2) + O(n^2) = O(n^{\log_2 7}) \approx O(n^{2.807})$$

$$T(n) = kT(n/2) + O(n^2) \implies O(n^{\log_2 k})$$

The exponent depends only on k, the number of block products. This is the lever part (b) tries to pull — 5 instead of 7 — for squaring.

Matrix squaring - five products, and why they don't recurse 2.27

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad A^2 = \begin{bmatrix} a^2 + bc & b(a + d) \\ c(a + d) & d^2 + bc \end{bmatrix}$$

Five products: a^2 , d^2 , bc , $b(a+d)$, $c(a+d)$.

The recursion fails:

$$A_{11}A_{12} + A_{12}A_{22} \neq A_{12}(A_{11} + A_{22})$$

$ab + bd = b(a + d)$ used commutativity. Matrices do not commute.

Also: $A_{12}A_{21}$ is a *product*, not a square - nothing to recurse on.

Matrix squaring - squaring $\equiv$ multiplying 2.27(c)

$$(A + B)^2 = A^2 + AB + BA + B^2 \implies AB + BA = (A + B)^2 - A^2 - B^2$$

Three squarings + $O(n^2)$.

Kill BA with a block embedding:

$$A = \begin{bmatrix} X & 0 \\ 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & Y \\ 0 & 0 \end{bmatrix} \implies AB + BA = \begin{bmatrix} 0 & XY \\ 0 & 0 \end{bmatrix}$$

$$S(n) = O(n^c) \implies \text{multiplication in } O(n^c)$$

1. Split at the median x into L and R .
2. Recurse; $d = \min(d_L, d_R)$.
3. Keep the strip $|x_i - x| < d$; sort it by y .
4. Compare each point with the next **seven**.
5. Best of the three candidate pairs.

Recursion handles $L-L$ and $R-R$ for free. All the work is in split pairs.

Closest pair of points - why seven

Packing lemma. A $d \times d$ square holds ≤ 4 points of L .

Quarter it: two points in one quarter would be $\leq d/\sqrt{2} < d$ apart.

Points between p and q in y -order lie in a $2d \times d$ rectangle = two $d \times d$ squares $\Rightarrow \leq 8$ points $\Rightarrow \leq 6$ in between.

$$T(n) = 2T(n/2) + O(n \log n) = O(n \log^2 n)$$

$$\text{pre-sort by } y \text{ once} \implies T(n) = 2T(n/2) + O(n) = O(n \log n)$$

$n + 1$ points determine a polynomial of degree $\leq n$ **uniquely**.

$$p(x) = \sum_i y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j} \quad (\text{Lagrange})$$

Shortcut here: $p(3) = p(5) = 0 \Rightarrow p(x) = (x - 3)(x - 5) q(x)$, $\deg q = 2$.

$$p(x) = -\frac{3}{4}x^4 + \frac{25}{3}x^3 - \frac{125}{4}x^2 + \frac{137}{3}x - 20$$

Read the data before choosing the method.